

Day 4: Quantization

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ The model of Day 3 uses 32 bits for each weight and each activation. A microcontroller pays for each bit with flash, RAM, time, and energy.
- ▶ Quantization stores the same model with 8 bits for each value. The model becomes four times smaller, and integer kernels run faster.
- ▶ The cost is a rounding error. You must know where the error comes from and how to keep it small.
- ▶ In the lab you quantize tensors by hand, quantize a small CNN, and compare the float model and the int8 model on the XIAOML Kit.

After this day you can:

- ▶ Derive the scale and the zero point of an affine quantizer.
- ▶ Apply post-training quantization with a representative dataset.
- ▶ Apply quantization-aware training.
- ▶ Explain per-tensor and per-channel quantization.
- ▶ Measure the effect of int8 quantization on size, accuracy, and latency.

Part 1 Number formats and the affine mapping

Exercise: quantize five numbers by hand and calculate the error

Part 2 Post-training quantization

Exercise: predict the result of a calibration set with one class

Part 3 Quantization-aware training and debugging

Exercise: read a per-layer error report and name the layer to keep in float

Each value of a model has a width

Format	Bits	Bytes	Model of Day 3	MobileNetV2
FP32	32	4	25 096 bytes	13.4 MB
FP16	16	2	12 548 bytes	6.7 MB
INT8	8	1	6274 bytes	3.3 MB

- ▶ The table gives the bytes of the weights: the number of parameters times the bytes for each value.
- ▶ The model of Day 3 with the raw window has 6274 parameters. MobileNetV2 has 3 504 872 parameters.
- ▶ The same factor applies to the activations in RAM. The arena becomes smaller with the weights.

Key point: Quantization changes the format of the values. It does not change the architecture: the layers and the number of parameters stay the same.

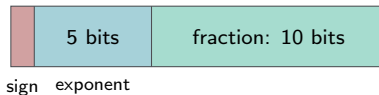


$$r = (-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{\text{exponent}-127}$$

- ▶ The exponent moves the binary point. So the format holds very small and very large numbers: from about 10^{-38} to 10^{38} .
- ▶ The precision is about 7 decimal digits, for a small number and for a large number.
- ▶ A training needs this range: gradients can be very small.

Key point: An inference needs much less. The weights and the activations of one layer are in a small range that you know after the training.

FP16: half the bits, the same idea



- ▶ The largest number is 65 504. The precision is about 3 decimal digits.
- ▶ The weights need half the bytes. The loss of accuracy is very small for almost all models.
- ▶ A microcontroller has no arithmetic for FP16. The runtime changes the weights to FP32 before it calculates.
- ▶ So FP16 saves flash. It saves no RAM and no time on a microcontroller.

Example

The LiteRT file of the Day 3 lecture has 27 020 bytes with FP32 weights and 15 248 bytes with FP16 weights.

Source: File sizes: results of the converter `onnx2tf 1.29.24`.

Floating point: the step grows with the number



Integer: the same step for all numbers



- ▶ An `int8` value is an integer from -128 to 127 : 256 values.
- ▶ The format has no exponent. The distance between two values is always 1.
- ▶ An integer alone cannot hold the weight 0.0173. You need a rule that connects the integers with real numbers.

Key point: The rule is the subject of this part: one scale and one zero point for each tensor.

Model of Day 3 with the raw window	FP32	INT8	Factor
Weights in the flash, in bytes	25 096	6274	4
Input tensor, 300 values, in bytes	1200	300	4
Peak of the activations, in bytes	1280	320	4

- ▶ The flash holds the weights. The arena holds the activations. The two become four times smaller.
- ▶ The complete file and the complete arena decrease by a smaller factor: the description of the graph and the data of the interpreter do not change.
- ▶ On Day 1, MobileNetV2 in `float32` did not fit the flash of the XIAO. In `int8` the weights need 3.3 MB.

Key point: For a microcontroller, the memory is the first reason for quantization. The speed is the second reason.

Operation	Width	Energy
Integer addition	8 bits	1
Float addition	32 bits	30
Read from memory	64 bits	40 000

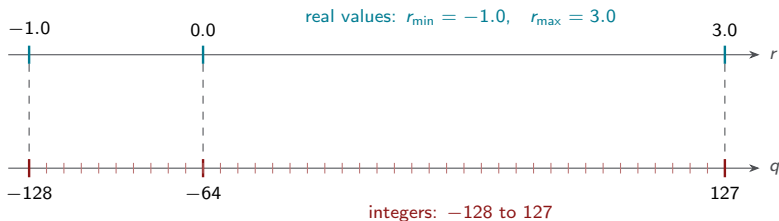
The unit is the energy of one integer addition with 8 bits.

- ▶ An integer operation needs a smaller circuit than a float operation.
- ▶ To move data costs much more than to calculate. With `int8`, each read moves a quarter of the bytes.
- ▶ Many small microcontrollers have no circuit for float arithmetic. A float operation is then a software function.

Key point: The gain in time depends on the device and on the kernels. You measure it on the board in Part D of the lab.

Source: Energy ratios: *Machine Learning Systems*, Volume I, chapter 10.

The idea: a ruler with 256 marks



- ▶ You know the range of a tensor: its smallest value $r_{\min}$ and its largest value $r_{\max}$.
- ▶ You put the 256 integers on this range, with the same distance between them.
- ▶ Each real value goes to the nearest mark.

Key point: Two numbers describe the ruler: the distance between two marks, and the integer that is at the real value 0.

From the integer to the real value

$$r = S(q - Z)$$

From the real value to the integer

$$q = \text{clamp}\left(\text{round}\left(\frac{r}{S}\right) + Z, -128, 127\right)$$

r The real value.

q The integer that the device stores.

S The scale: a positive real number. It is the real distance between two integers.

Z The zero point: the integer that represents the real value 0.

clamp Limits the result to the range of `int8`.

“Affine” means a multiplication and a shift. One tensor has one S and one Z . The model file stores them with the tensor.

Source: Mapping of Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”, 2018.

The range $[r_{\min}, r_{\max}]$ must go to the range $[-128, 127]$.

Scale

255 steps cover the range:

$$S = \frac{r_{\max} - r_{\min}}{255}$$

Zero point

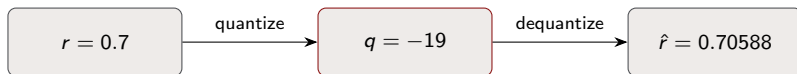
$r_{\min}$ must give $q = -128$:

$$Z = \text{round}\left(-128 - \frac{r_{\min}}{S}\right)$$

Example: $r_{\min} = -1.0$ and $r_{\max} = 3.0$. $S = 4.0/255 = 0.015686$,
 $r_{\min}/S = -63.75$, $Z = \text{round}(-128 + 63.75) = \text{round}(-64.25) = -64$.

- ▶ The zero point is an integer. So the real value 0 has an exact integer, with no rounding error.
- ▶ This is necessary: a padding with zeros and the output of a ReLU must be exactly 0.

With $S = 0.015686$ and $Z = -64$:



Quantize

$$0.7 / 0.015686 = 44.63$$

round: 45

$$q = 45 + (-64) = -19$$

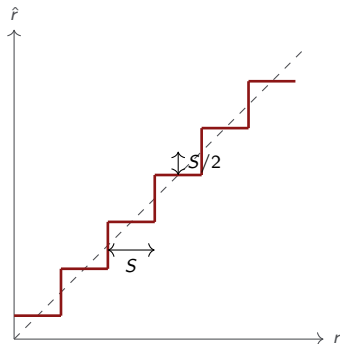
Dequantize

$$\hat{r} = 0.015686 \times (-19 + 64)$$

$$\hat{r} = 0.015686 \times 45 = 0.70588$$

- ▶ The error is $\hat{r} - r = 0.00588$. The step is 0.015686.
- ▶ The device stores -19 in one byte. It does not store 0.7.

Key point: The mapping loses information. You cannot get 0.7 back from -19 .

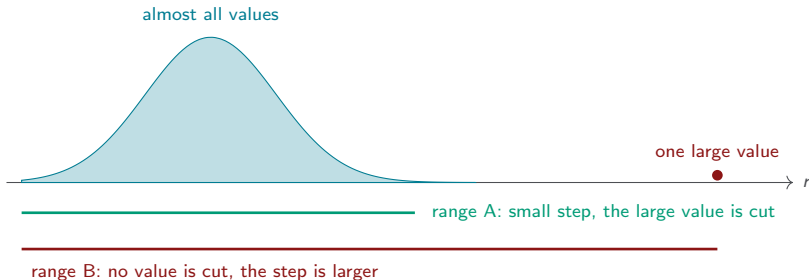


- ▶ Inside the range, each value goes to the nearest mark.
- ▶ The error is between $-S/2$ and $+S/2$.
- ▶ The typical error is $S/\sqrt{12}$, about $0.29 S$.
- ▶ One more bit makes S half as large. The error becomes half as large also.

Example

Range $[-1.0, 3.0]$ with 8 bits: $S = 0.0157$, largest error 0.0078. The same range with 4 bits: $S = 4/15 = 0.267$, largest error 0.133.

Key point: A small scale gives a small rounding error. The scale is small when the range is small.



- ▶ A value outside the range goes to -128 or to 127 . This is clipping. Its error has no limit.
- ▶ Range A cuts the large value, and all other values have a small rounding error.
- ▶ Range B keeps the large value, and all other values have a larger rounding error.

Key point: The selection of the range is a trade-off between the two errors. Part 2 shows how a calibration set finds the range.

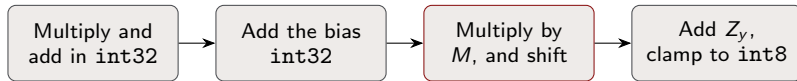
	Symmetric	Asymmetric
Real range	$[-a, +a]$	$[r_{\min}, r_{\max}]$
Zero point	$Z = 0$	Any integer from -128 to 127
Integers	-127 to 127	-128 to 127
Used for	Weights	Activations
Reason	The weights are around 0. With $Z = 0$, the multiplication is more simple.	After a ReLU, all values are positive. A symmetric range uses only half of the integers.

Example: an activation after a ReLU with the range $[0, 6]$.

- ▶ Asymmetric: $S = 6/255 = 0.0235$ and $Z = -128$. All 256 integers are in use.
- ▶ Symmetric: $S = 6/127 = 0.0472$. The integers below 0 are never in use, and the step is two times as large.

Source: Rules of the `int8` quantization specification of LiteRT.

A fully connected layer calculates $y = \sum_i w_i x_i + b$. Each real value has its mapping: $w = S_w q_w$, $x = S_x(q_x - Z_x)$, $y = S_y(q_y - Z_y)$.



$$q_y = Z_y + M \left(\sum_i q_{w,i} (q_{x,i} - Z_x) + q_b \right) \quad M = \frac{S_w S_x}{S_y}$$

- ▶ The sum of many products of `int8` values needs more than 8 bits. The kernel uses a sum with 32 bits.
- ▶ M is a real number below 1. The converter stores it as an integer and a shift. The device then needs no float operation.
- ▶ The result goes back to 8 bits. The next layer gets `int8` values again.

Source: Jacob et al., 2018.

- ▶ The bias adds to the sum of the products $q_w (q_x - Z_x)$.
- ▶ One unit of this sum is the real value $S_w \times S_x$.
- ▶ So the bias uses this scale, with the zero point 0:

$$S_b = S_w \times S_x \quad q_b = \text{round}\left(\frac{b}{S_b}\right)$$

- ▶ S_b is very small. The integer q_b is then large: it needs 32 bits.

The model of the Day 3 exercise

The first layer of `hello_world.tflite`, as Netron shows it:

Input scale	0.024574
Weight scale	0.004224
Bias scale	0.00010381
$0.024574 \times 0.004224 =$	
0.00010381	

Key point: This answers the question of Day 3: the weights have 8 bits, and the biases have 32 bits. A model has few biases, so they cost little flash.

Source: Scale values: the file `hello_world.tflite` of `Labs/hardware/HW-02/`.

```
TfLiteTensor* input = interpreter->input(0);
TfLiteTensor* output = interpreter->output(0);

// The model has int8 input. Quantize the real value x.
const float S_in = input->params.scale;
const int Z_in = input->params.zero_point;
input->data.int8[0] = (int8_t)(roundf(x / S_in) + Z_in);

interpreter->Invoke();

// The model has int8 output. Dequantize the result.
const float S_out = output->params.scale;
const int Z_out = output->params.zero_point;
const float y = S_out * (output->data.int8[0] - Z_out);
```

- ▶ The tensor holds its scale and its zero point. The sketch reads them from the model file.
- ▶ The sketch does the first step and the last step of the mapping. All steps between them are integer steps of the kernels.
- ▶ For a classifier, the largest output is the class. You can compare the int8 outputs directly, with no dequantization.

Source: The code follows the sketch `tflm_hello` of Labs/hardware/HW-02/.

The three formats in comparison

	FP32	FP16	INT8
Bytes for a value	4	2	1
Values	About 4 billion	65 536	256
Step	Grows with the number	Grows with the number	Constant: the scale
Flash	Baseline	Half	A quarter
RAM on a microcontroller	Baseline	No change	A quarter
Arithmetic	Float	Float, after a change to FP32	Integer
Loss of accuracy	None	Very small	Small. You must measure it.

Key point: The format `int8` is the standard for a microcontroller: it decreases the flash, the RAM, and the time. FP16 decreases only the flash.

Exercise: quantize five numbers by hand

A tensor has the range $r_{\min} = -1.00$ and $r_{\max} = 1.55$. The target format is `int8`: from -128 to 127 .

1. Calculate the scale S and the zero point Z .
2. Quantize these five values. Then dequantize them, and calculate the error $\hat{r} - r$.

0.000 0.500 -0.333 1.234 2.000

3. Which value has the largest error? Why?
4. What is the largest rounding error for a value inside the range?

$$S = \frac{r_{\max} - r_{\min}}{255} \quad Z = \text{round}\left(-128 - \frac{r_{\min}}{S}\right)$$
$$q = \text{clamp}\left(\text{round}\left(\frac{r}{S}\right) + Z\right) \quad \hat{r} = S(q - Z)$$

Answer: quantize five numbers by hand

1. $S = 2.55/255 = 0.01$ and $Z = \text{round}(-128 + 100) = -28$.

2. The table:

r	$\text{round}(r/S)$	q	$\hat{r}$	Error
0.000	0	-28	0.00	0.000
0.500	50	22	0.50	0.000
-0.333	-33	-61	-0.33	0.003
1.234	123	95	1.23	-0.004
2.000	200	127 (clamp)	1.55	-0.450

3. The value 2.000. It is outside the range. The sum $200 - 28 = 172$ is above 127, so the clamp gives 127. This is the clipping error.

4. $S/2 = 0.005$. The errors 0.003 and -0.004 are below this limit. The values 0.000 and 0.500 are exactly on a mark.

Part 1 Number formats and the affine mapping

Exercise: quantize five numbers by hand and calculate the error

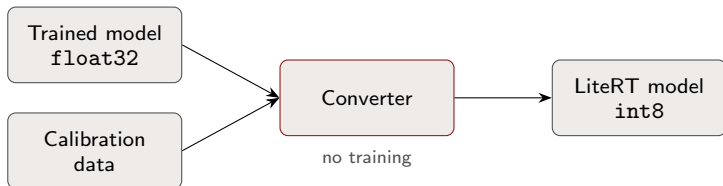
Part 2 Post-training quantization

Exercise: predict the result of a calibration set with one class

Part 3 Quantization-aware training and debugging

Exercise: read a per-layer error report and name the layer to keep in float

Quantize a model that is trained



- ▶ Post-training quantization (PTQ) changes the format of a model after the training.
- ▶ It needs no training loop, no labels, and no loss function. It needs some seconds.
- ▶ The converter must know the range of each tensor. It then calculates the scale and the zero point of Part 1.

Key point: Always try post-training quantization first. Part 3 gives the method for a model that loses too much accuracy.

Weights

- ▶ They are in the model file.
- ▶ They do not change after the training.
- ▶ The converter reads the smallest and the largest weight of each tensor. It needs no data.
- ▶ This difference gives two methods: one quantizes only the weights, the other quantizes the weights and the activations.

Activations

- ▶ They exist only when the model runs.
- ▶ They depend on the input.
- ▶ The converter must run the model with real inputs and record the range of each activation.

Key point: The range of an activation is a property of the model and of its data together.

Method	Weights	Activations	Data	File
None	float32	float32	None	26 732
Float16	float16	float32	None	14 764
Dynamic range	int8	float32	None	9056
Full integer	int8	int8	Calibration set	8680

- ▶ The file sizes are in bytes, for the model of Day 3 with 300 inputs and 6274 parameters.
- ▶ The weights go from 25 096 bytes to 6274 bytes. The file does not become four times smaller, because the description of the graph stays.
- ▶ Only the full integer model has int8 activations. Only this model makes the arena smaller.

Source: File sizes: results of the converter of TensorFlow 2.21.0.

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
```

- ▶ One line more than the float conversion. No data is necessary.
- ▶ The weights are `int8` in the file. The input, the output, and the activations stay `float32`.
- ▶ At run time, the kernel finds the range of each activation for each input. From this comes the name “dynamic range”.

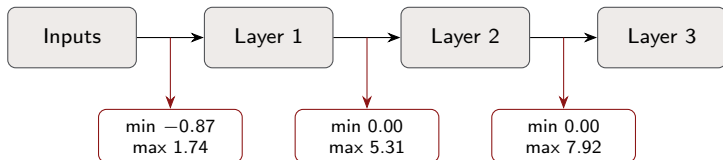
Good for	A phone or a Raspberry Pi: the file is smaller, and the sketch or the program needs no change
Not good for	A microcontroller: the arena does not become smaller, and the kernels still use float arithmetic

Source: The arena of the dynamic range model is equal to the arena of the float model: 1232 bytes for the feature model of Day 3, in a 32-bit build of TensorFlow Lite Micro.

```
def representative_dataset():  
    for i in range(200):  
        yield [x_calibration[i:i + 1]]          # one input, with batch size 1  
  
converter = tf.lite.TFLiteConverter.from_keras_model(model)  
converter.optimizations = [tf.lite.Optimize.DEFAULT]  
converter.representative_dataset = representative_dataset  
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]  
converter.inference_input_type = tf.int8  
converter.inference_output_type = tf.int8  
tflite_model = converter.convert()
```

- ▶ `representative_dataset`: the converter runs the model with these inputs and records the range of each activation.
- ▶ `TFLITE_BUILTINS_INT8`: each operator must have an `int8` kernel. If one operator has none, the conversion stops with an error.
- ▶ `inference_input_type`: the model takes `int8` input. The sketch quantizes the input, as Part 1 showed.

Calibration: record the range of each activation



1. The converter puts an observer at each activation tensor.
2. It runs the float model for each input of the calibration set.
3. Each observer keeps the smallest and the largest value that it saw.
4. From each range, the converter calculates one scale and one zero point.

Key point: The calibration is a measurement of the model with your data. It trains nothing and changes no weight.

What it must be

- ▶ Inputs of the same type as the inputs on the device.
- ▶ With the same pre-processing as the training.
- ▶ From all classes and all conditions.
- ▶ 100 to 500 inputs are usually enough.
- ▶ A range that is too small cuts values on the device: clipping.
- ▶ A range that is too large makes the step large for all values: rounding error.

What it does not need

- ▶ Labels.
- ▶ Many inputs: the range of a tensor is stable after some hundreds.
- ▶ The test set. Take the inputs from the training set.

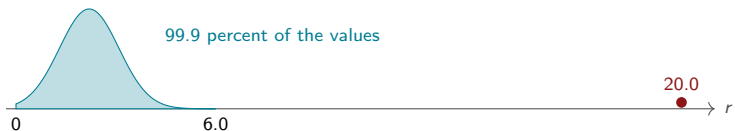
Key point: The calibration set decides the range. A wrong calibration set gives a wrong model, and the conversion shows no error.

Method	The range is	Weak point
Minimum and maximum	The smallest and the largest value that the observer saw	One extreme value makes the range large
Moving average	The mean of the minimum and of the maximum over the batches	Depends on the order and the size of the batches
Percentile	The range that holds 99.9 percent of the values	Cuts the rare, large values
Minimum error	The range that gives the smallest difference between the float values and the quantized values	Needs more calculation

- ▶ The LiteRT converter records the minimum and the maximum.
- ▶ Other tools let you select the method. Day 7 uses a second tool.

Source: Method list: *Machine Learning Systems*, Volume I, chapter 10; Nagel et al., “A White Paper on Neural Network Quantization”, 2021.

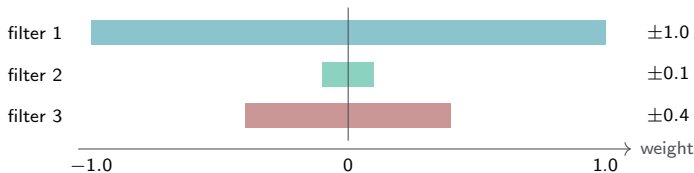
One extreme value changes the scale



Range	Scale	Largest rounding error	Clipping
Minimum and maximum: [0, 20]	0.0784	0.0392	None
Percentile: [0, 6]	0.0235	0.0118	20 becomes 6

- ▶ With the full range, each of the normal values has a rounding error that is 3.3 times as large.
- ▶ With the percentile range, one value in a thousand has a large clipping error. You measure which range gives the better accuracy.

Per-tensor quantization of a convolution



- ▶ A convolution has one filter for each output channel. The weights of different filters can have very different sizes.
- ▶ Per-tensor: one scale for all filters. The largest filter decides: $S = 1.0/127 = 0.00787$.
- ▶ Filter 2 has weights up to 0.1. With this scale, its integers are between -13 and 13 . It uses 27 of the 255 levels.

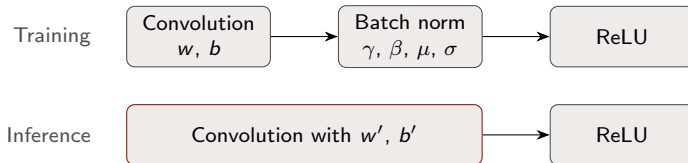
Key point: With one scale for the tensor, a filter with small weights has a large relative error.

	Largest weight	Per-tensor scale	Per-channel scale	Levels in use
Filter 1	1.0	0.00787	0.00787	255 and 255
Filter 2	0.1	0.00787	0.00079	27 and 255
Filter 3	0.4	0.00787	0.00315	103 and 255

- ▶ Per-channel: each filter has its own scale. Each filter then uses all levels.
- ▶ The cost is small: one scale for each output channel in the file, and one multiplier M for each channel in the kernel.
- ▶ The zero point of the weights stays 0 for all channels.
- ▶ The activations stay per-tensor: one scale and one zero point for each activation tensor.

Key point: The LiteRT converter uses per-channel scales for the weights of convolution layers. You see the list of the scales in Netron.

Source: The last column gives the levels with the per-tensor scale and with the per-channel scale.
Rules: int8 quantization specification of LiteRT.



$$w' = w \cdot \frac{\gamma}{\sigma} \qquad b' = \beta + (b - \mu) \cdot \frac{\gamma}{\sigma}$$

- ▶ At inference, a batch normalization is a multiplication and an addition with constants, for each channel.
- ▶ The converter puts these constants into the weights and the bias of the convolution before it. The layer disappears.
- ▶ The folding must come before the quantization. The device runs w' , so the range of w' decides the scale.
- ▶ The factor γ/σ is different for each channel. So the folded filters have very different ranges. This is the main reason for per-channel scales.

The input and the output of the quantized model

	int8 input and output	float32 input and output
Setting of the converter	Input type and output type <code>tf.int8</code>	The default
Operators in the file	Only the operators of the model	One QUANTIZE and one DEQUANTIZE more
The sketch	Quantizes the input with the scale and the zero point of the input tensor	Writes float values, as on Day 3
Input tensor in the arena	1 byte for each value	4 bytes for each value

- The output of a softmax in `int8` always has the scale $1/256$ and the zero point -128 . The integer -128 is the probability 0, and 127 is the probability 0.996.

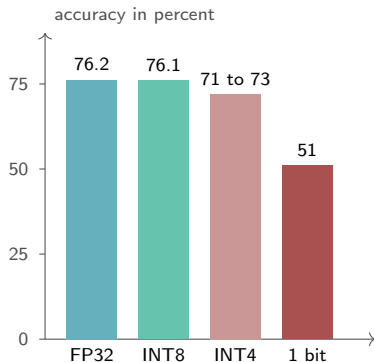
Source: Softmax scale: the full integer model of this part, and the `int8` quantization specification of LiteRT.

Check the quantized model on the laptop

```
interpreter = Interpreter(model_content=tflite_model)
interpreter.allocate_tensors()
inp = interpreter.get_input_details()[0]
out = interpreter.get_output_details()[0]
S, Z = inp["quantization"]                # scale and zero point

correct = 0
for x, label in zip(x_test, y_test):
    q = np.clip(np.round(x / S) + Z, -128, 127).astype(np.int8)
    interpreter.set_tensor(inp["index"], q[None, :])
    interpreter.invoke()
    correct += interpreter.get_tensor(out["index"])[0].argmax() == label
print(correct / len(x_test))
```

- ▶ The LiteRT interpreter on the laptop uses the same integer arithmetic as the device.
- ▶ Compare the accuracy of the float model and of the int8 model on the same test set.
- ▶ Count the input values that the clamp changes. Many clipped values show a calibration problem.



- ▶ The figure shows ResNet-50 on ImageNet.
- ▶ With 8 bits, most models lose less than 1 percentage point.
- ▶ Below 8 bits, the accuracy decreases fast.
- ▶ A small model has less redundancy than a large model. It can lose more.

Rule for the next step

Loss below 1 point: use the PTQ model. Loss of 1 to 3 points: try quantization-aware training. Loss above 3 points: find the layer that causes the error (Part 3).

Source: Numbers and rule: slides of *Machine Learning Systems*, Volume I, chapter 10.

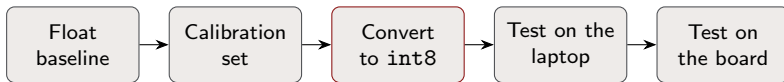
Feature model of Day 3 (63 inputs)	float32	int8
Peak of the activations, in bytes	332	83
Arena used, in bytes	1232	1028

- ▶ The activations become four times smaller: from $63 \times 4 + 80 = 332$ bytes to $63 + 20 = 83$ bytes.
- ▶ The arena decreases by 204 bytes only. For this very small model, the data of the interpreter is the largest part of the arena.
- ▶ For a model with images, the activations are the largest part. Then the arena decreases by a factor near 4.

Key point: Know which part of a budget is large before you optimize it. The gain of a method depends on the model.

Source: Arena values: 32-bit build of TensorFlow Lite Micro. The values of the board can be different. You measure them in the lab.

The steps of post-training quantization



1. **Float baseline.** Measure the accuracy, the file size, and the latency of the float model.
2. **Calibration set.** Take 100 to 500 inputs from the training set, from all classes.
3. **Convert.** Full integer quantization with `int8` input and output.
4. **Test on the laptop.** The same test set as for the float model. Compare.
5. **Test on the board.** Flash, arena, latency, and the class for real inputs.

Key point: Each number of the `int8` model needs the same number of the float model next to it.

Exercise: a calibration set with one class

A team quantizes a motion classifier. The model takes the raw window: 300 acceleration values in g . The classes have these value ranges:

	idle	terrestrial	lift	maritime
Smallest value	0.00	-0.87	-0.08	-0.52
Largest value	1.01	1.13	1.74	1.39

The team takes the calibration set from one folder. The folder holds only recordings of the class "idle".

1. Which range does the converter record for the input tensor?
Calculate the scale.
2. What happens to a value of -0.87 , and to a value of 1.74 ?
3. Predict the accuracy for each class: no change, or lower?
4. The conversion gives no error. How can the team find the problem?

1. The range of the class “idle”: about $[0.00, 1.01]$. The scale is $1.01/255 = 0.0040$.
2. The two values are outside the range. The clamp changes -0.87 to 0.00 and 1.74 to 1.01 . The model receives a window that the sensor did not measure.
3. “idle”: no change. The motion classes: lower, because a part of their values is cut.
4. Count the clipped input values on the test set, and compare the accuracy of each class with the float model.

Calibration set	Total	idle	terr.	lift	mar.
All classes: accuracy	100.0	100.0	100.0	100.0	100.0
Only “idle”: accuracy	69.2	100.0	59.5	100.0	17.4
Only “idle”: clipped values		0.0	34.7	28.5	44.7

All values are in percent, for the fallback dataset of Day 2 (simulated signals) with the converter of TensorFlow 2.21.0. The class “lift” keeps its accuracy with 28.5 percent of clipped values: the effect depends on the class.

Part 1 Number formats and the affine mapping

Exercise: quantize five numbers by hand and calculate the error

Part 2 Post-training quantization

Exercise: predict the result of a calibration set with one class

Part 3 Quantization-aware training and debugging

Exercise: read a per-layer error report and name the layer to keep in float

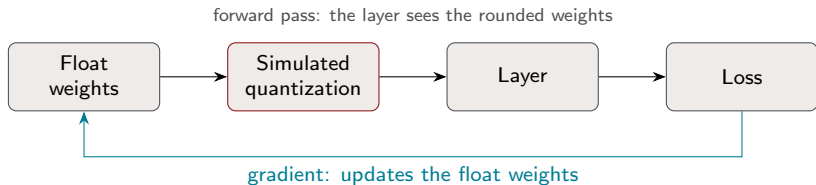
- ▶ The weights of a float model were trained for float arithmetic. Nobody told the training about the rounding.
- ▶ PTQ adds the rounding error after the training. The model cannot react to it.
- ▶ For most models with 8 bits, the error is small enough.

Cases with a larger loss

- ▶ A very small model: it has no redundancy.
- ▶ Less than 8 bits.
- ▶ A layer with a very wide range of values.
- ▶ An output that must be exact, for example a regression.

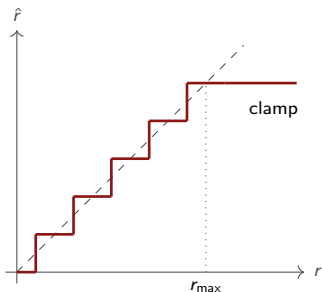
Key point: Two answers exist. Find the cause and repair it (debugging), or let the training learn weights that accept the rounding (quantization-aware training).

The idea of quantization-aware training



- ▶ The training keeps the weights in `float32`.
- ▶ In the forward pass, each weight and each activation goes through the mapping of Part 1: quantize, then dequantize.
- ▶ The loss then includes the rounding error. The optimizer finds weights that give a small loss *with* the rounding.
- ▶ After the training, the converter makes a real `int8` model.

Key point: The model trains with the error that the device will have.

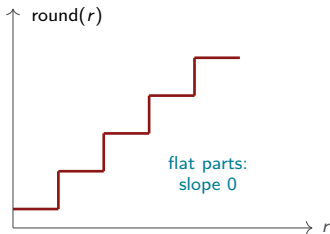


$$q = \text{clamp}\left(\text{round}\left(\frac{r}{S}\right) + Z\right) \quad \hat{r} = S(q - Z)$$

- ▶ The result $\hat{r}$ is a float value. It can have only 256 different values.
- ▶ The other name is “fake quantization”: the value has the error of an integer, and the type of a float.
- ▶ A normal training framework can calculate with it. It needs no integer kernels.

Key point: The forward pass of the training has the same rounding error as the `int8` model on the device.

The problem: rounding has no gradient



- ▶ Training needs the gradient of the loss for each weight.
- ▶ The round function is a staircase. On each step it is flat: the slope is 0.
- ▶ With the chain rule, a slope of 0 makes the complete gradient 0.
- ▶ The weights get no update. The training stops.

Key point: A small change of a weight does not change its rounded value. So the exact gradient says: no direction is better.

Pass	The rounding is
Forward	The real staircase
Backward	The identity: slope 1

The gradient goes “straight through” the rounding.

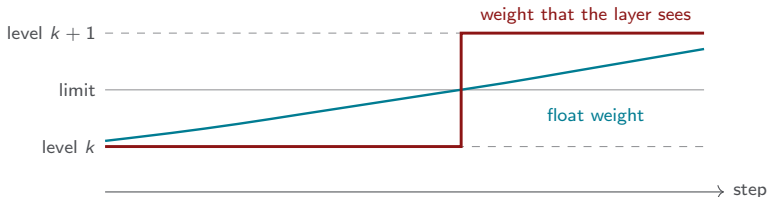
```
from keras import ops

def fake_quant(x, S, Z):
    q = ops.clip(ops.round(x / S) + Z,
                 -128, 127)
    x_q = S * (q - Z)
    # forward: x_q, backward: identity
    d = ops.stop_gradient(x_q - x)
    return x + d
```

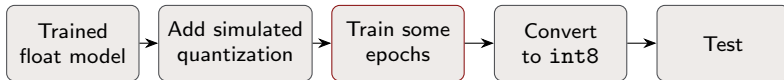
- ▶ The estimator is not the correct gradient. It is a useful approximation: “if the float weight grows, the rounded weight grows also, in the end”.
- ▶ `stop_gradient` hides the difference $x_q - x$ from the backward pass. The forward value is still x_q .

Source: Bengio et al., “Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation”, 2013.

Float weights collect small updates



- ▶ One optimizer step changes a weight by much less than one quantization step.
- ▶ The float weight keeps the sum of the small updates.
- ▶ When the float weight passes the limit between two levels, the weight that the layer sees changes by one step.
- ▶ An integer weight with no float copy cannot do this: a small update disappears in the rounding.



- ▶ **Start from the trained float model.** Do not start from random weights.
- ▶ **Use a small learning rate,** for example a tenth of the rate of the float training. Some epochs are enough.
- ▶ **The ranges** of the activations come from the training data: an observer keeps a moving average of the minimum and the maximum.
- ▶ **Convert** with the same settings as PTQ. Then test the real `int8` model, not the simulated model.

Key point: Quantization-aware training is a fine-tuning step after the normal training. It does not replace it.

Compare the two methods

	Post-training (PTQ)	Quantization-aware (QAT)
Data	Some hundreds of inputs, no labels	The training set, with labels
Time	Seconds	A part of the training time
You need	The converter	The training code
Accuracy	Good for most models with 8 bits	Recovers 0.5 to 1.0 points of the loss
Use it	Always first	When PTQ misses the target, or below 8 bits

- ▶ The two methods give the same type of file: an `int8` LiteRT model. The device sees no difference.
- ▶ A different model, with fewer problems for the quantizer, is sometimes less work than QAT.

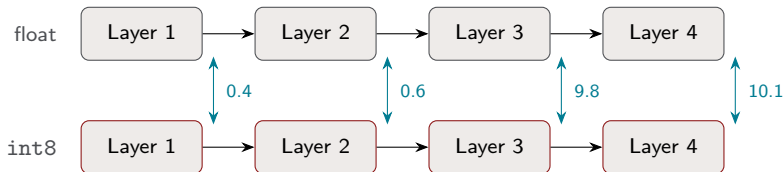
Source: Accuracy range: slides of *Machine Learning Systems*, Volume I, chapter 10.

First checks when the accuracy decreases

Check	Typical cause
The float LiteRT model	If this model is wrong, the problem is the conversion or the test code, not the quantization.
The input of the test	The test code does not quantize the input, or it uses a wrong scale or a wrong zero point.
The pre-processing	The calibration set has a different normalization than the training set.
The clipped inputs	The calibration set does not cover all classes (exercise of Part 2).
The accuracy of each class	One class fails. The total accuracy hides it.

Key point: Most “quantization problems” are data problems or code problems. Do these five checks before you change the method.

Find the layer: compare the outputs of each layer



1. Run the float model and the `int8` model with the same inputs.
 2. After each layer, dequantize the `int8` output and compare it with the float output.
 3. Write the error in percent of the range of the float output.
- The error of a layer includes the errors of all layers before it. Look for the layer where the error grows suddenly: layer 3 in the figure.

Find the layer: quantize one layer at a time

Model	Accuracy	Loss
All layers in float	92.0	none
Only layer 1 in int8	91.9	0.1
Only layer 2 in int8	91.8	0.2
Only layer 3 in int8	87.5	4.5
Only layer 4 in int8	91.9	0.1
All layers in int8	87.0	5.0

- ▶ This second method measures the accuracy directly. It needs one conversion and one test for each layer.
- ▶ The table is the sensitivity of the model: which layer costs how much accuracy.
- ▶ One layer usually causes most of the loss. The sum of the single losses is near the total loss.

Layer	Reason	Possible repair
The first layer	The input has a wide range, or values that the calibration set did not have	A better calibration set, or a normalized input
A depthwise convolution	After the folding, the channels have very different ranges	Per-channel scales
A layer with ReLU	The activation has no upper limit. Some values are very large.	ReLU6: the range is always 0 to 6
An addition of two branches	The two inputs have different scales	Quantization-aware training
The last layer	Small differences between the outputs decide the class	Keep this layer in float

Key point: Many repairs are changes of the model, not of the quantizer. A model that is designed for quantization loses less.



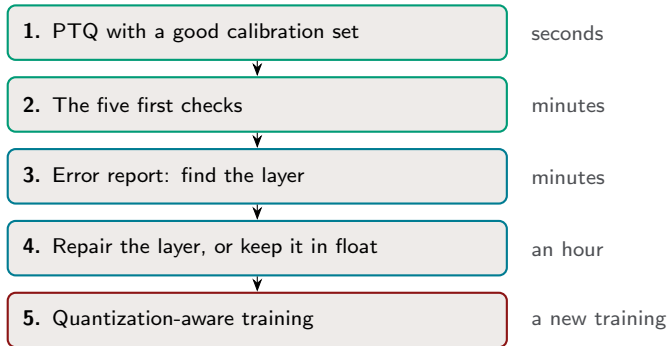
- ▶ Mixed precision: different layers use different formats. The sensitive layer keeps more bits.
- ▶ The converter adds one operator before and one operator after the float layer.
- ▶ **Cost:** the weights of the float layer need four times the flash, its activations need four times the RAM, and the device needs a float kernel for it.
- ▶ A second option of LiteRT: activations with 16 bits and weights with 8 bits.

Key point: Keep in float only the layer that the error report names. Then measure the size and the accuracy again.

	INT8	Ternary	Binary
Values of a weight	256	-1, 0, +1	-1, +1
Bits	8	2	1
Size against FP32	1/4	1/16	1/32
Multiplication	Integer multiplier	Add, subtract, or skip	XNOR gate
Training	PTQ or QAT	QAT	QAT

- ▶ The loss of accuracy is large. ResNet-50 on ImageNet: 76.2 percent in FP32, and 51 percent with 1 bit.
- ▶ The gain needs kernels or hardware that calculate with single bits. TensorFlow Lite Micro has no such kernels.
- ▶ Use these formats only when `int8` does not fit. Compare with a smaller `int8` model first.

Source: *Machine Learning Systems*, Volume I, chapter 10, and its slides.



- ▶ Stop at the first step that reaches your accuracy target.
- ▶ Each step costs more time than the step before it.

Key point: Write the target before you start: for example, “the int8 model loses less than 1 point”.

Part	Time	Work
A	30 min	Quantize tensors by hand in NumPy. Measure the rounding error and the clipping error.
B	50 min	Quantize a small CNN after the training. Change the calibration set, and observe the accuracy.
C	40 min	Train the same CNN with simulated quantization. Compare with the result of Part B.
D	30 min	Put the float model and the <code>int8</code> model of Day 3 on the board. Compare flash, arena, latency, and accuracy.

- ▶ The deliverable is a comparison table with one float row and one `int8` row for the size, the accuracy, and the latency.
- ▶ The Decision Log explains the change of the accuracy.

Key point: Each `int8` number needs the float number next to it, from the same test set and the same board.

Exercise: read a per-layer error report

A small CNN has an accuracy of 91.0 percent in float and 85.2 percent in int8. The target is a loss below 1 point. The error report:

Layer	Type	Parameters	Output error	Loss
1	Convolution	448	0.5	0.1
2	Depthwise convolution	160	7.9	4.9
3	Pointwise convolution	544	8.2	0.3
4	Fully connected	5130	8.4	0.2

“Output error”: the difference between the float output and the int8 output after the layer, in percent of the range. “Loss”: the loss of accuracy in points with only this layer in int8.

1. Which layer do you keep in float? Give two reasons from the table.
2. Layers 3 and 4 have a large output error. Why do they stay in int8?
3. How many bytes of flash does your decision cost?
4. Name one repair that you try before you keep the layer in float.

Answer: read a per-layer error report

1. Layer 2, the depthwise convolution. First reason: the output error grows from 0.5 to 7.9 percent at this layer. Second reason: this layer alone costs 4.9 points of the 5.8 points.
2. Their error comes from layer 2. The error of a layer includes the errors of all layers before it. Alone, layers 3 and 4 cost 0.3 and 0.2 points.
3. About 480 bytes. The 160 parameters need 160 bytes in `int8` and 640 bytes in `float32`. The exact value is 432 bytes: the 16 biases have 32 bits in the two formats. Add the two operators DEQUANTIZE and QUANTIZE. The activations of this layer also need four times the RAM.
4. Per-channel scales for the depthwise convolution, if the converter did not use them. Or ReLU6 in place of ReLU after the layer. Or quantization-aware training.

The expected result: a loss of about $5.8 - 4.9 = 0.9$ points. This is below the target of 1 point. Measure it.

- ▶ An affine quantizer maps a real range to integers with two numbers: the scale and the zero point. The rounding error is at most half of the scale.
- ▶ Post-training quantization needs no training. Full integer quantization needs a representative dataset to find the range of each activation.
- ▶ Quantization-aware training simulates the rounding in the forward pass, so the weights adapt to it.
- ▶ When the accuracy decreases, find the layer that causes the error before you change the method.

Key point: With int8, a model needs one quarter of the memory. Measure the accuracy, the size, and the latency of the float model and of the int8 model on the same test set and on the same board.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 10 “Model Compression”, section “Quantization and Precision”.

Papers

- ▶ “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”, Jacob et al., 2018.
- ▶ “A White Paper on Neural Network Quantization”, Nagel et al., 2021.
- ▶ “Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation”, Bengio et al., 2013 (the straight-through estimator).

Tools

- ▶ LiteRT documentation, model optimization:
<https://ai.google.dev/edge/litert>
- ▶ Netron, to read the scale and the zero point of each tensor:
<https://netron.app>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.